BORANA UNIVERSITY

- COLLEGE OF ENGINEERING AND TECHNOLOGY
- DEPARTMENT OF COMPUTER SCIENCE

Advanced Database System

CHAPTER TWO (2)
QUERY PROCESSING AND OPTIMIZATION

Query processing

- It refers to the range of activities involved in extracting data from a database.
- It takes various steps for fetching the data from the database.
- It is activities that involved in parsing , validating, execution and optimizing a query

- Aims of QP:

- Transform query written in high-level language (e.g. SQL query), into correct and efficient execution strategy expressed in low-level language (implementing RA); execute strategy to retrieve required data.

Cont....

- ***There are three steps involved in query processing***

1, Parsing and translation

✔ The parser checks the syntax and also verifies the relation names in the query matches relations in database

✔The system construct a parse tree representation of the query.

✔The parse-tree is used to form a relational algebra expression.

2, Query optimization

✔ The relational algebra expression can be evaluated by many methods or ways.

✔ Optimization involves the best way to evaluate the query with lowest cost.

3, Query Evaluation: the evaluation engine takes a selected query evaluation plan, execute the plan and finally provides output to the users

Cont..

Query evaluation plan

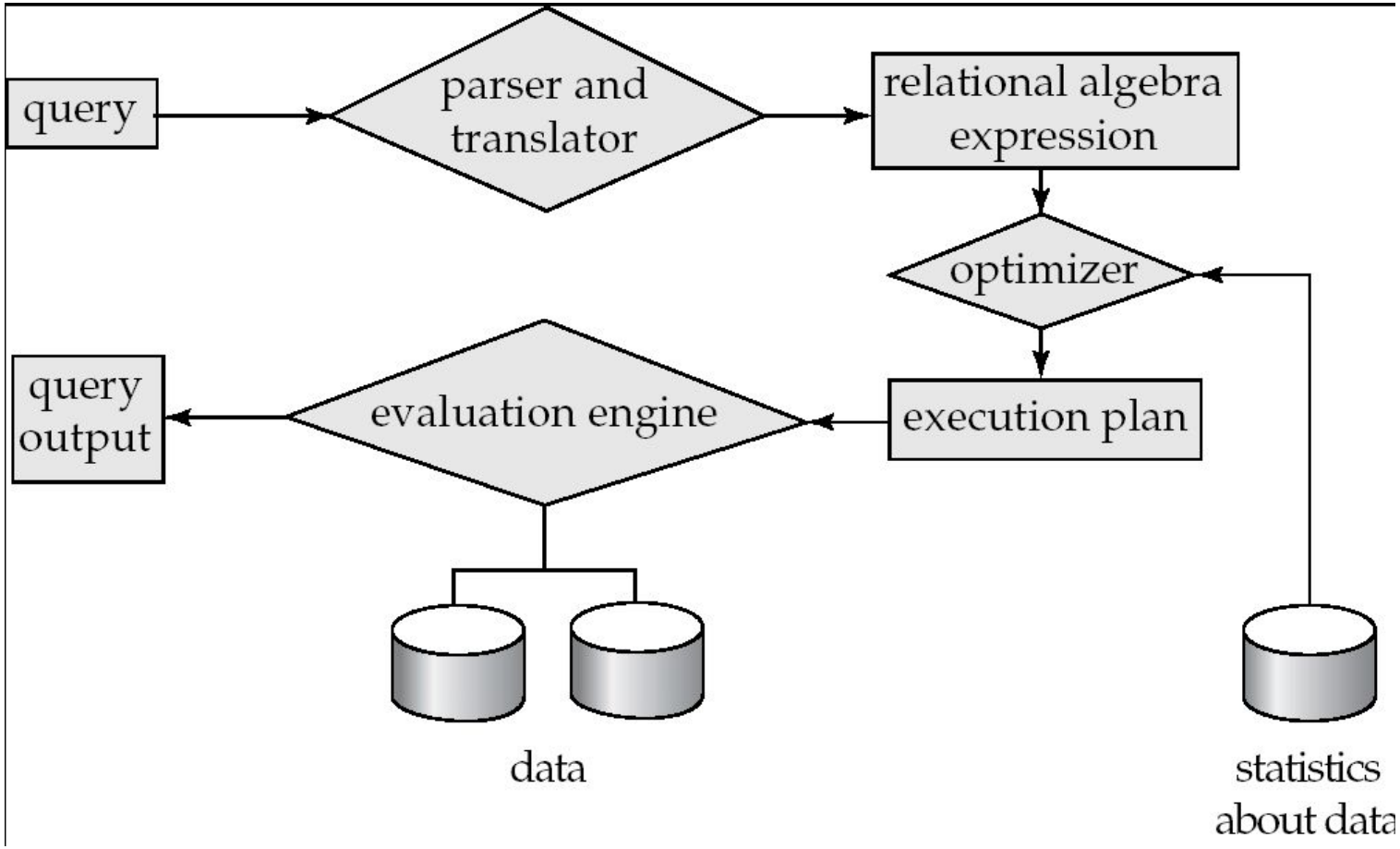
- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in the evaluation plan may refer to the algorithms to be used for the particular index or the specific operations.
- Such relational algebra with annotations is referred to as Evaluation Primitives.
- The evaluation primitives carry the instructions needed for the evaluation of the operation.
- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query.
- The query evaluation plan is also referred to as the query execution plan.

Cont..

- A query execution engine is responsible for generating the output of the given query.
- It takes the query execution plan, executes it, and finally makes the output for the user query.
- Finally, after selecting an evaluation plan, the system evaluates the query and produces the output of the query.

Cont….

.


query
parser and translator
relational algebra expression
optimizer
query output
evaluation engine
execution plan
data
statistics about data

cont..

Example:

SQL Query:

select balance from account where balance < 2500;

This query can be translated into either of the following relational-algebra expressions:

- $\sigma_{balance<2500}$ ($\Pi_{balance}$ (account))
- $\Pi_{balance}$ ($\sigma_{balance<2500}$ (account))

Translating SQL Queries into Relational Algebra(RA)

- An SQL query is first translated into an equivalent extended relational algebra expression—represented as a query tree data structure—that is then optimized
- SQL queries are decomposed into query blocks, which form the basic units that can be translated into the algebraic operators and optimized.
- **Query block:** basic unit that can be translated into the algebraic operators and optimized.
- A query block contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clause if these are part of the block.
- **Nested queries** within a query are identified as separate query blocks.
- **Aggregate operators**(such as: sum, min, average, max) in SQL must be included in the extended algebra.

Cont..

- Consider the following SQL query on the relation name EMPLOYEE:

  SELECT Lname, Fname FROM EMPLOYEE WHERE Salary > (SELECT MAX (Salary)FROM EMPLOYEE WHERE Dno=5);

- This query retrieves the names of employees (from any department in the company) who earn a salary that is greater than the *highest salary in department 5*.

- The query includes a nested sub query and hence would be decomposed into two blocks.

- The inner block is:

  (SELECT MAX (Salary) FROM EMPLOYEE WHERE Dno=5)

- This retrieves the highest salary in department 5.

Cont..

- The outer query block is:

  SELECT Lname, Fname FROM EMPLOYEE WHERE Salary > c

- where c represents the result returned from the inner block. The inner block could be translated into the following extended relational algebra expression:

  $\Im$MAX Salary(σDno=5(EMPLOYEE))

- And the outer block into the expression: - πLname,Fname(σSalary>c(EMPLOYEE))

Translating SQL…..

```
SELECT    LNAME, FNAME
FROM      EMPLOYEE
WHERE     SALARY > C(   SELECT    MAX (SALARY)
                FROM      EMPLOYEE
                WHERE     DNO = 5);
```

```
SELECT    LNAME, FNAME
FROM      EMPLOYEE
WHERE     SALARY > C
```

$$\pi_{\text{LNAME, FNAME}}(\sigma_{\text{SALARY>C}}(\text{EMPLOYEE}))$$

```
SELECT    MAX (SALARY)
FROM      EMPLOYEE
WHERE     DNO = 5
```

$$\mathcal{F}_{\text{MAX SALARY}}(\sigma_{\text{DNO=5}}(\text{EMPLOYEE}))$$

- The query optimizer would then choose an execution plan for each block.

Cont..

- The *query optimizer* would then choose an execution plan for each query block.
- Notice that in the above example, the inner block needs to be evaluated only once to produce the maximum salary of employees in department 5, which is then used—as the constant c—by the outer block.
- We called this a *nested query (without correlation with the outer query)*.

Query Decomposition

- Aims are to transform high-level query into RA query and check that query is syntactically and semantically correct.
- Typical stages are:
  - Analysis:
    - ✓ Analyze query lexically and syntactically using compiler techniques.
    - ✓ Verify relations and attributes exist.
    - ✓ Verify operations are appropriate for object type.
  - Normalization:
    - ✓ Converts query into a normalized form for easier manipulation.

cont…

- Semantic analysis:
  - ✓ Rejects normalized queries that are incorrectly formulated or contradictory.
- Simplification:
  - ✓ Detects redundant qualifications,
  - ✓ eliminates common sub-expressions,
  - ✓ transforms query to semantically equivalent but more easily and efficiently computed form.
- Query restructuring:
  - Restructure the query appropriate for optimization.

Basic algorithms for executing query operations

Sorting

- Sorting is one of the primary algorithms used in query processing.
- For example, whenever an SQL query specifies an ORDER BY-clause, the query result must be sorted.
- Sorting is also a key component in sort-merge algorithms used for JOIN and other operations (such as UNION and INTERSECTION), and in duplicate elimination algorithms for the PROJECT operation (when an SQL query specifies the DISTINCT option in the SELECT clause).

cont….

- External sorting refers to sorting algorithms that are suitable for large files of records stored on disk that do not fit entirely in main memory, such as most database files.
- The typical external sorting algorithm uses a sort-merge strategy, which starts by sorting small sub-files—called runs—of the main file and then merges the sorted runs, creating larger sorted sub-files that are merged in turn.
- The sort-merge algorithm, like other database algorithms, requires buffer space in main memory, where the actual sorting and merging of the runs is performed.

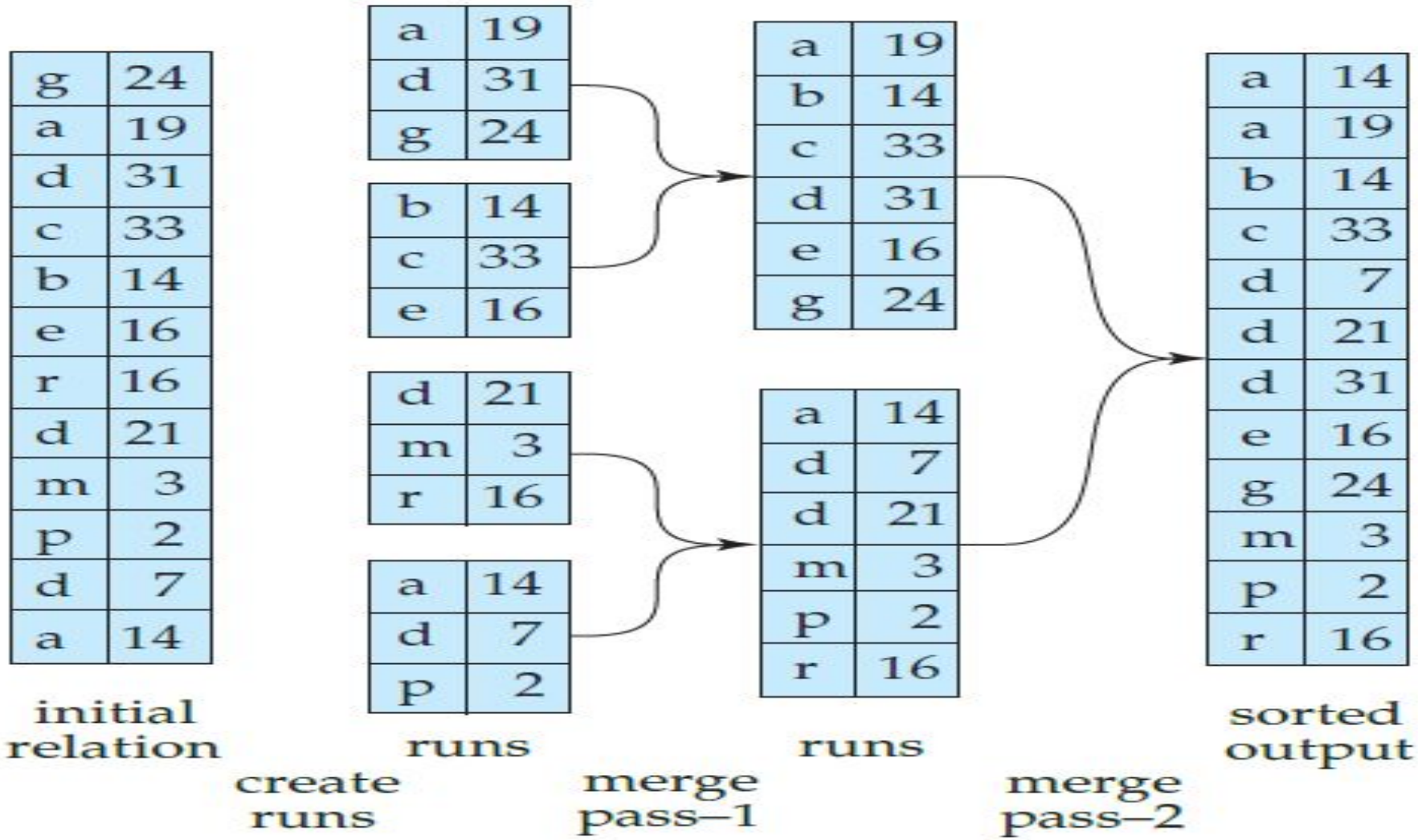
cont….

- The basic algorithm consists of two phases: (1) the sorting phase and (2) the merging phase.
- In the sorting phase, runs (portions or pieces) of the file that can fit in the available buffer space are read into main memory, sorted using an internal sorting algorithm, and written back to disk as temporary sorted sub-files (or runs).
- The size of a run and number of initial runs (n_R) is dictated by the number of file blocks (b) and the available buffer space (n_B).

Cont...

- In the merging phase, the sorted runs are merged during one or more merge passes.
- Each merge pass can have one or more merge steps.
- The degree of merging (*dM*) is the number of sorted sub-files that can be merged in each merge step.
- During each merge step, one buffer block is needed to hold one disk block from each of the sorted sub-files being merged, and one additional buffer is needed for containing one disk block of the merge result, which will produce a larger sorted file that is the result of merging several smaller sorted sub-files.

Cont...


g 24
a 19
d 31
c 33
b 14
e 16
r 16
d 21
m 3
p 2
d 7
a 14
initial relation
a 19
d 31
g 24
b 14
c 33
e 16
d 21
m 3
r 16
a 14
d 7
p 2
runs
create runs
a 19
b 14
c 33
d 31
e 16
g 24
a 14
d 7
d 21
m 3
p 2
r 16
runs
merge pass–1
a 14
a 19
b 14
c 33
d 7
d 21
d 31
e 16
g 24
m 3
p 2
r 16
sorted output
merge pass–2

Algorithm for search operation

- The following search methods (S1 through S5) are examples of some of the search algorithms that can be used to implement a select operation:
- S1. In a linear search, the system scans each file block and tests all records to see whether they satisfy the selection condition.
  - ✔For a selection on a key attribute, the system can terminate the scan if the required record is found, without looking at the other records of the relation.

Cont….

- S2 (*Binary search)*: If the file is ordered on an attribute, and the selection condition is an equality comparison on the attribute, we can use a binary search to locate records that satisfy the selection.

- The system performs the binary search on the blocks of the file.

- Implementing the SELECT Operation

- Examples:
  - (OP1): $\sigma_{SSN='123456789'}$(EMPLOYEE)
  - (OP2): $\sigma_{DNUMBER>5}$(DEPARTMENT)
  - (OP3): $\sigma_{DNO=5}$(EMPLOYEE)
  - (OP4): $\sigma_{DNO=5 \text{ AND } SALARY>30000 \text{ AND } SEX=F}$(EMPLOYEE)
  - (OP5): $\sigma_{ESSN=123456789 \text{ AND } PNO=10}$(WORKS_ON)

Cont....

- S3a—Using a primary index. If the selection condition involves an equality comparison on a key attribute with a primary index—for example, Ssn = '123456789' in OP1—use the primary index to retrieve the record.
- Note that this condition retrieves a single record (at most).
- S3b—Using a hash key. If the selection condition involves an equality comparison on a key attribute with a hash key—for example, Ssn = '123456789' in OP1—use the hash key to retrieve the record.
- Note that this condition retrieves a single record (at most).

Cont...

S4—Using a primary index to retrieve multiple records.

- If the comparison condition is >, >=, <, or <= on a key field with a primary index—for example, Dnumber > 5 in OP2—use the index to find the record satisfying the corresponding equality condition (Dnumber = 5), then retrieve all subsequent records in the (ordered) file.
- For the condition Dnumber < 5, retrieve all the preceding records.

S5—Using a clustering index to retrieve multiple records.

- If the selection condition involves an equality comparison on a non-key attribute with a clustering index—for example, Dno = 5 in OP3—use the index to retrieve all the records satisfying the condition.

Algorithm for Project and set operations

Project Operation

- A *Project* operation π<attribute list>(*R*) is straightforward to implement if <attribute list> includes a key of relation *R*.
- Because in this case the result of the operation will have the same number of tuples as *R*, but with only the values for the attributes in <attribute list> in each tuple.
- If <attribute list> does not include a key of *R*, *duplicate tuples must be eliminated*.
- This can be done by sorting the result of the operation and then eliminating duplicate tuples, which appear consecutively after sorting.

Cont….

Set operations:

Union, Intersection, Set difference, and Cartesian product—are sometimes expensive to implement.

- CARTESIAN PRODUCT of relations R and S include all possible combinations of records from R and S. The attribute of the result include all attributes of R and S.

  CARTESIAN PRODUCT operation is very expensive and should be avoided if possible

- UNION

  ✔Sort the two relations on the same attributes.

  ✔Scan and merge both sorted files concurrently, whenever the same tuple exists in

Cont....

INTERSECTION

- ✔Sort the two relations on the same attributes.
- ✔Scan and merge both sorted files concurrently, keep in the merged results only those tuples that appear in both relations.

SET DIFFERENCE R-S

- ✔Keep in the merged results only those tuples that appear in relation R but not in relation S.

Algorithm for JOIN operation

- The *Join* operation is one of the most time-consuming operations in query processing.
- There are many possible ways to implement a *two-way join*, which is a join on two files.
- Joins involving more than two files are called multi-way joins.
- The number of possible ways to execute multi-way joins grows very rapidly.

- Implementing the JOIN Operation:
  - Join (EQUIJOIN, NATURAL JOIN)
    - two–way join: a join on two files
    - e.g. $R_{A=B} S$
    - multi-way joins: joins involving more than two files.
    - e.g. $R_{A=B} S_{C=D} T$
- Examples
  - (OP6): $EMPLOYEE_{DNO=DNUMBER} DEPARTMENT$
  - (OP7): $DEPARTMENT_{MGRSSN=SSN} EMPLOYEE$

Implementing Aggregate Operations and Outer Joins

Outer Join Operators:

LEFT OUTER JOIN: Returns all records from the left table and the matched record s from the right table

RIGHT OUTER JOIN: Returns all records from the right table and the matched record s from the left table

FULL OUTER JOIN: Returns all records when there is a match in either left or right table. Example:

```
SELECT    FNAME, DNAME
FROM (EMPLOYEE LEFT OUTER JOIN DEPARTMENT
ON DNO = DNUMBER);
```

Note: The result of this query is a table of employee names and their associated departments. It is similar to a regular join result, with the exception that if an employee does not have an associated department, the employee's name will still appear in the resulting table, although the department name would be indicated as null.

Methods for implementing joins.

- *J1—Nested-loop join (or nested-block join)*
  - This is the default (brute force) algorithm, as it does not require any special access paths on either file in the join.
  - For each record t in R (outer loop), retrieve every record s from S (inner loop) and test whether the two records satisfy the join condition *t [A] = s[B].*
- *J2—Single-loop join* (using an access structure to retrieve the matching records).
  - *If an index (or hash key)* exists for one of the two join attributes— say, attribute B of file S—retrieve each record t in R (loop over file R), and then use the access structure (such as an index or a hash key) to retrieve directly all matching records s from S that satisfy *s[B] = t[A].*

Query Optimization

- **Query Optimization:** is the process of choosing a suitable execution strategy for processing a query.
- It is the activity of choosing an efficient execution strategy for processing a query.
- As there are many equivalent transformations of same high-level query, the aim of QO is .
  - ✔ to choose one that minimizes resource usage
  - ✔ reduce total execution time of query.
  - ✔ reduce response time of query.
- Two main techniques for implementing query optimization:

✔heuristic based query optimizer

✔Cost based query optimizer

Using heuristics in query optimization

- The parser of a high-level query first generates an *initial internal representation*,
- which is usually in the form of a *query tree* or a *query graph* data structure, which is then optimized according to heuristic rules.
- Following that, a *query execution plan* is generated to execute groups of operations based on the access paths available on the files involved in the query.

cont...

- The main heuristic is to apply first the operations that reduce the size of intermediate results i.e.
- it apply Select and Project operations before applying the Join or other binary operations.
- This is because the size of the file resulting from a binary operation—such as Join—is usually a multiplicative function of the sizes of the input files.
- The Select and Project operations reduce the size of a file and hence should be applied before a join or other binary operation.

cont

❖Query Tree:

- ✔A tree data structure that corresponds to a **relational algebra expression**. It represents the input relations of the query as leaf nodes of the tree, and represents the relational algebra operations as internal nodes
- ✔An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation
- ✔the query tree represents a specific order of operations for executing a query.
- ✔The execution terminates when the root node is executed and produces the result relation for the query.

cont...

❖**Query Graph:**

- ✔A **graph data structure** that corresponds to a relational calculus expression.
- ✔It does not indicate an order on which operations to perform first. There is only a single graph corresponding to each query

Example:

- ✔For every project located in ‘Stafford’, retrieve the project number, the controlling department number and the department manager’s last name, address and birthdate

Cont..

- SQL Query (Q2):

```
SELECT P.NUMBER, P.DNUMBER, E.LNAME, E.ADDRESS,
E.BDATE FROM PROJECT AS P, DEPARTMENT AS D,
EMPLOYEE AS E WHERE P.DNUM=D.DNUMBER AND
D.MGRSSN=E.SSN AND P.PLOCATION='STAFFORD';
```

- **Relation Algebra:**

$$\pi_{\text{PNUMBER, DNUM, LNAME, ADDRESS, BDATE}}(((\sigma_{\text{PLOCATION='STAFFORD'}}(\text{PROJECT}) \bowtie_{\text{DNUM=DNUMBER}} (\text{DEPARTMENT})) \bowtie_{\text{MGRSSN=SSN}} (\text{EMPLOYEE}))$$

Cont...

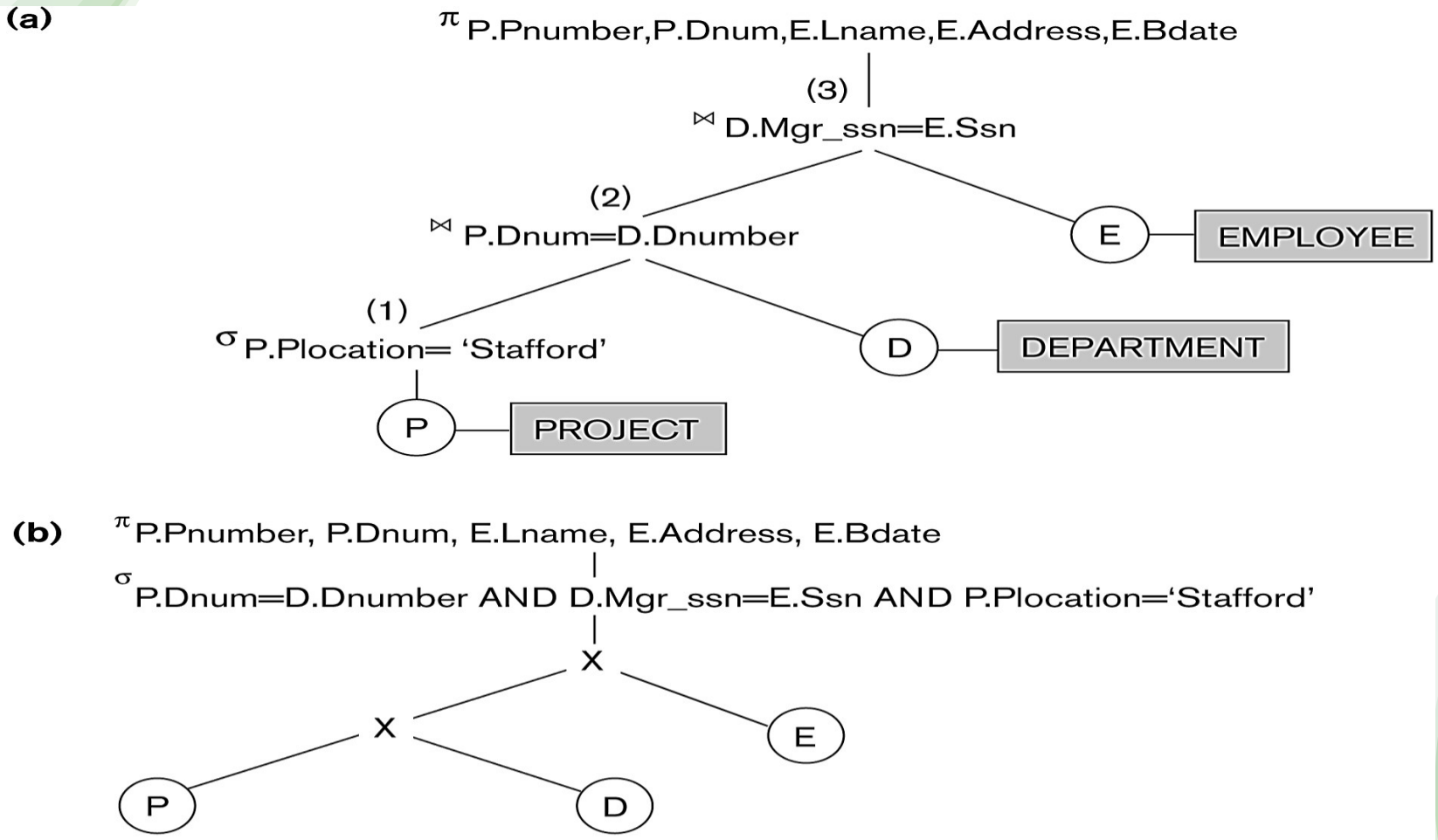


Figure 15.4
Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Cont….

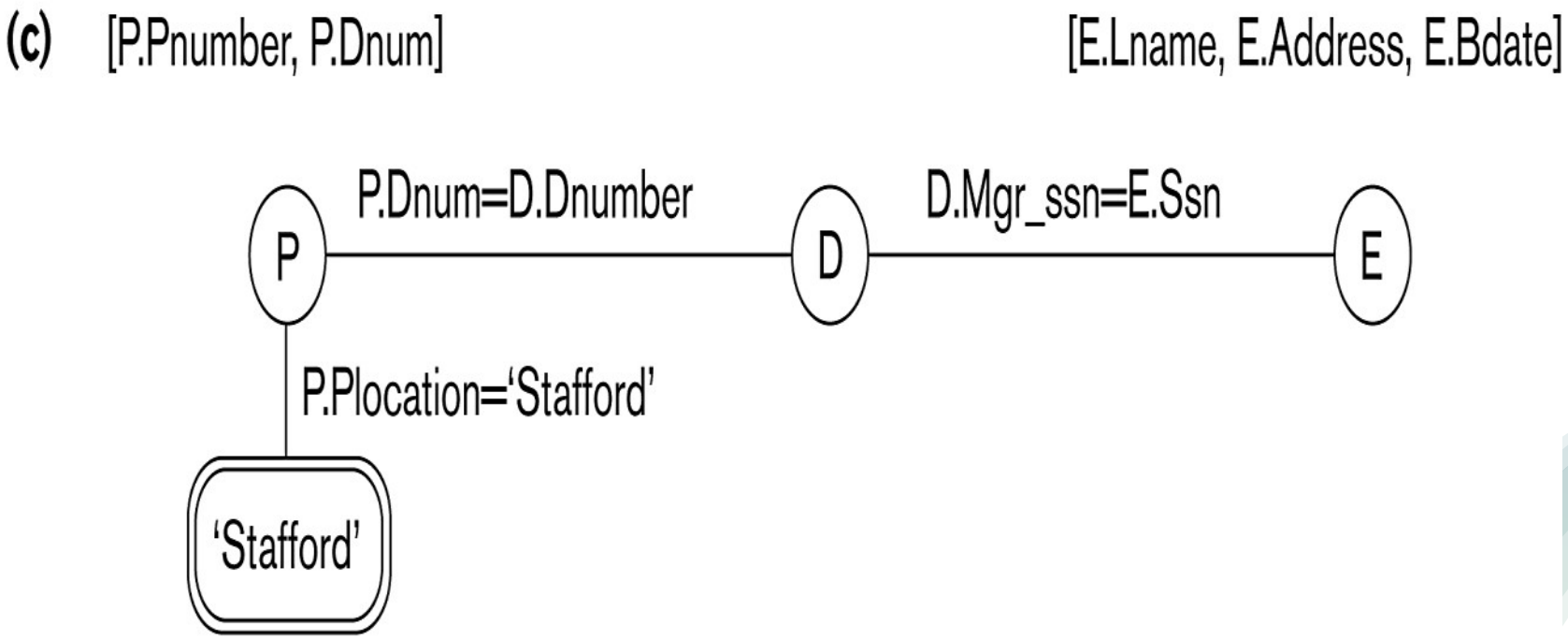


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Cont..

Heuristic optimization of query trees

- ✔ The **same query** could correspond to many different relational algebra expressions - and hence many different query trees
- ✔ A query tree can be transformed step by step into another query tree that is more efficient to execute.
- ✔ The task of heuristic optimization of query trees is to find a **final query tree** that is efficient to execute.

Cont...

- Steps to converting a query tree during query optimization to apply Heuristic query optimization
  1) initial(canonical) query tree for SQL query
  2) Moving SELECTION operation down the query tree
  3) Applying the more restrictive SELECT operation first
  4) Replacing CARTESIAN PRODUCT and SELECT with JOIN operation .
  5) Moving PROJECT operation down the query tree.

Example: "Find the last names of employees born after 1957 who work on a project named 'Aquarius'."

```
SELECT LNAME
FROM   EMPLOYEE, WORKS_ON, PROJECT
WHERE  PNAME = 'AQUARIUS' AND
       PNMUBER=PNO AND ESSN=SSN
       AND BDATE > '1957-12-31';
```

Cont....

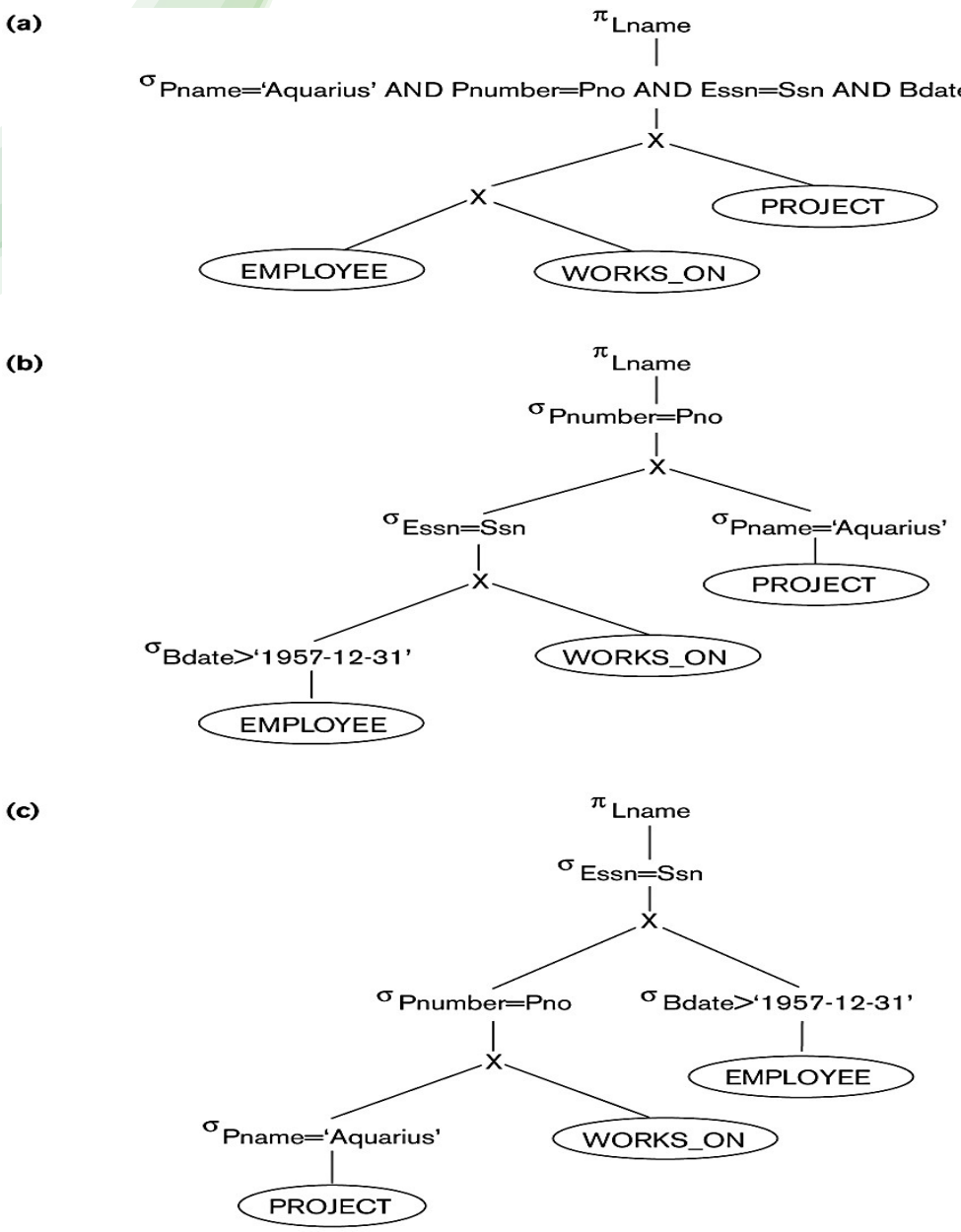
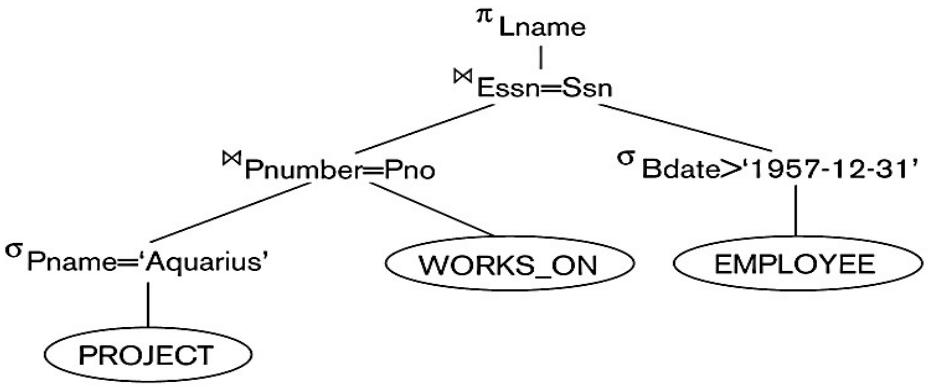


Figure 15.5
Steps in converting a query tree during heuristic optimization.
(a) Initial (canonical) query tree for SQL query Q.
(b) Moving SELECT operations down the query tree.
(c) Applying the more restrictive SELECT operation first.
(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.
(e) Moving PROJECT operations down the query tree.

Cont….

(d)



(e)

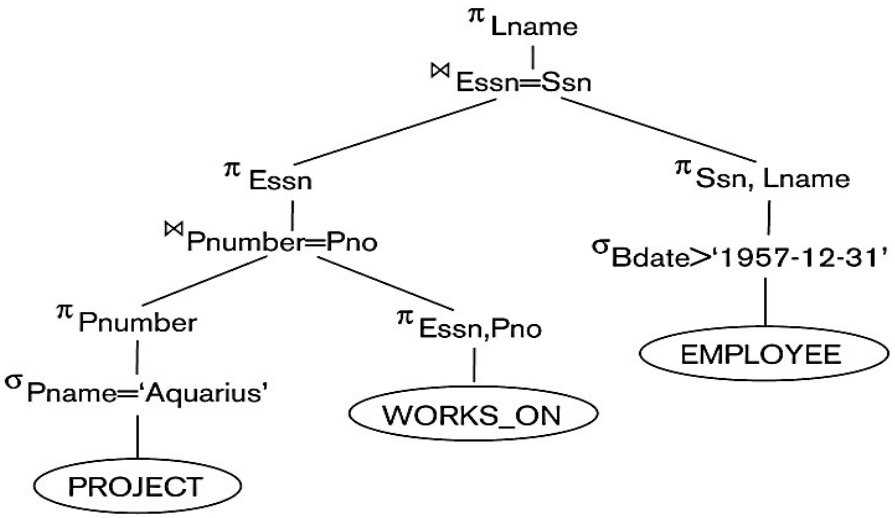


Figure 15.5
Steps in converting a query tree during heuristic optimization.
(a) Initial (canonical) query tree for SQL query Q.
(b) Moving SELECT operations down the query tree.
(c) Applying the more restrictive SELECT operation first.
(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.
(e) Moving PROJECT operations down the query tree.

Using selectivity and cost estimates in query optimization

- A query optimizer does not depend solely on heuristic rules; it also estimates and compares the costs of executing a query using different execution strategies and algorithms, and it then chooses the strategy with the *lowest cost estimate*.
- For this approach to work, accurate *cost estimates* are required so that different strategies can be compared fairly and realistically.
- In addition, the optimizer must limit the number of execution strategies to be considered; otherwise, too much time will be spent making cost estimates for the many possible execution strategies.
- Hence, this approach is more suitable for *compiled queries* where the optimization is done at compile time and the resulting execution strategy code is stored and executed directly at runtime.

Cost components for query execution

- The cost of executing a query includes the following components:

1. *Access cost to secondary storage.*

- This is the cost of transferring (reading and writing) data blocks between secondary disk storage and main memory buffers.
- This is also known as *disk I/O (input/output) cost*.

2. Disk storage cost.

- This is the cost of storing on disk any intermediate files that are generated by an execution strategy for the query.

Cont...

3. Computation cost.

- This is the cost of performing in-memory operations on the records within the data buffers during query execution.
- Such operations include searching for and sorting records, merging records for a join or a sort operation, and performing computations on field values.
- It is also known as CPU (central processing unit) cost.

4. Memory usage cost.

- This is the cost pertaining to the number of main memory buffers needed during query execution.

5. Communication cost.

- This is the cost of shipping the query and its results from the database site to the site or terminal where the query originated.

Semantic query optimization

- Uses constraints specified on the database schema in order to modify one query into another query that is more efficient to execute
- Consider the following SQL query,

  SELECT E.LNAME, M.LNAME FROM EMPLOYEE AS E, WHERE E.SUPERSSN=M.SSN AND E.SALARY>M.SALARY.

  - Suppose that we had a constraint on the database schema that stated that no employee can earn more than his or her direct supervisor.
- If the semantic query optimizer checks for the existence of this constraint, it need not execute the query at all because it knows that the result of the query will be empty.
- Techniques known as theorem proving can be used for this purpose.

THANKS!

QUESTION ?